In Ruby, many binary operators are implemented as method calls. For example, when you write `a * b + c`, you're actually asking the object referenced by `a` to execute the method *, passing in the parameter `b`. You then ask the object that results from that calculation to execute the + method, passing `c` as a parameter. This is the same as writing the following (perfectly valid) Ruby:

```
a, b, c = 1, 2, 3
a * b + c     # => 5
(a.*(b)).+(c) # => 5
```

Because everything is an object and because you can redefine instance methods, you can always redefine basic arithmetic if you don't like the answers you're getting:

```
class Integer
  alias old_plus +

  # Redefine addition of Integers. This is a BAD IDEA!
  def +(other)
    old_plus(other).succ
  end
end

1 + 2     # => 4
a = 3
a += 4    # => 8
a + a + a # => 26
```

What's going on here? First off, we're re-opening the `Integer` class to allow new definitions inside it, a feature of the Ruby object model that we talked about in Reopening Classes, on page 49. Inside the class, the keyword `alias` allows you to give a new name to an existing method, here we rename the + method to `old_plus`. The syntax here might seem odd, there's just a space between the new name and the old name. Typically, you'd use `alias` because you are planning on overwriting the existing method but you want that new method to still be able to access the original, as in this case, we've re-written + to use `old_plus` but add 1.

More usefully, classes you write can participate in operator expressions just as if they were built-in objects. For example, the left shift operator, `<<`, is often used to mean *append to receiver*. Arrays support this:

```
a = [1, 2, 3]
a << 4    # => [1, 2, 3, 4]
```

You can add similar support to your classes:

```
class ScoreKeeper
  def initialize
    @total_score = @count = 0
  end

  def <<(score)
    @total_score += score
    @count += 1
    self
  end

  def average
    fail "No scores" if @count.zero?
```